ENGINEERING DEMO HANDBOOK

Angular Microfrontends

Complete application flow and code-level explanation

Project A / Project B / Project C

Native Federation - RxJS - SharedWorker - cross-tab messaging

A detailed, scenario-based guide to the actual application: startup, remote loading, every custom function, data ownership, observable updates, Angular rendering and failure handling.

Includes architecture diagrams, message traces, demo guidance and complete numbered custom-source listings.

Prepared from the workspace source on 14 October 2026.

Contents

01 | What this application actually implements 3

02 | Files, dependencies and running from scratch 5

03 | Startup: from npm start to the first Angular view 7

04 | Native Federation: configuration and remote URL resolution 9

05 | The data contract and message protocol 11

06 | AppStateClient: every field and function 12

07 | RxJS internals in this application 15

08 | The A-origin bridge: every callback 17

09 | SharedWorker: every function and business rule 19

10 | Angular feature code: every project and template 21

11 | Scenario: add, edit, complete and delete one task 23

12 | Scenario: counters, simultaneous tabs and acknowledgement order 25

13 | Scenario: late join, reload, closing A and failures 27

14 | A practical code walkthrough and debugging route 29

15 | Complete source reference and reading instructions 30

16 | References and verification scope 45

Page numbers and PDF bookmarks are clickable. Chapters 6, 8, 9 and 10 explain functions; chapters 11-13 trace real scenarios. The source appendix follows chapter 15.

01 | What this application actually implements

This handbook explains the current Angular-DEMO source as inspected on 14 October 2026. It follows executable code, not a hypothetical architecture. Read chapters 1-5 for the startup and federation path, chapters 6-10 for the state and messaging implementation, and chapters 11-15 for user actions, concurrency, failures, and the live demo. The final appendix includes the complete custom source with original line numbers.

The three applications

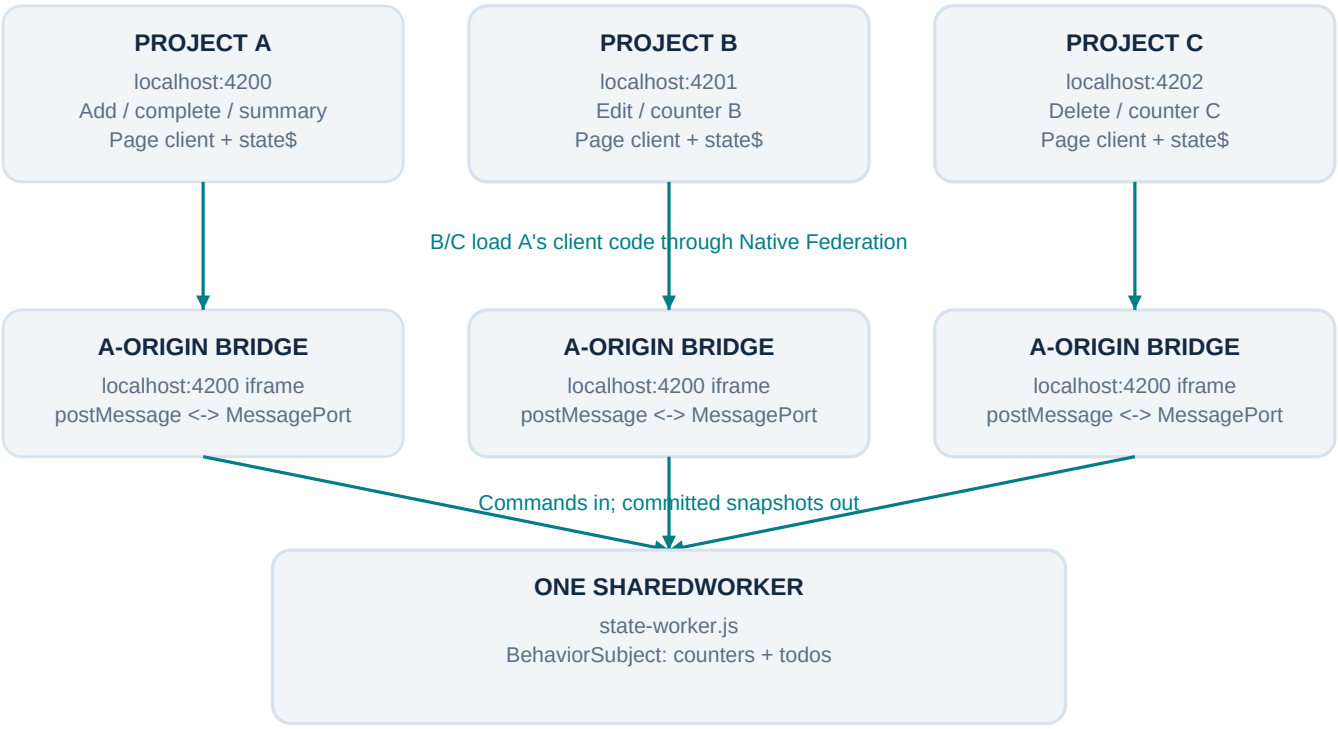
Project A runs at http://localhost:4200. It creates tasks, toggles completion, displays both counters, and supplies the federated state-client module. Project B runs at http://localhost:4201. It edits task text and changes counter B. Project C runs at http://localhost:4202. It deletes tasks and changes counter C. Each project bootstraps its own Angular application in its own tab.

The visible workflow is: create in A, edit in B, optionally complete in A, delete in C. All applications render the same business snapshot. A shows both counters; B and C display their own counter. These are UI responsibilities, not authenticated permissions enforced by a backend.

Three mechanisms working together

Native Federation delivers executable code. B and C download A's exposed AppState module at runtime and obtain its exported appState client. RxJS represents state updates as observable emissions. Browser messaging carries commands and snapshots between separate JavaScript environments. A SharedWorker is the single owner of the business state.

The distinction matters: sharing a module implementation does not make its objects shared across browser tabs. A module evaluated in B creates an object in B. The same module evaluated in C creates an object in C. A worker is needed because the requirement is one in-memory authority serving separate tabs.



What is common and what is local?

There is one authoritative BehaviorSubject in state-worker.js for the connected browser context. Each page has an AppStateClient, a latest-view cache, an observable replay buffer, a pending request identifier, and its own hidden iframe. Each iframe has a MessagePort to the worker. B's editing draft and A's input text are local UI values until submitted.

The worker owns countB, countC and todos. The client adds connected, pending and error. A pending edit in B does not make C pending. C may submit a different action while B waits. All committed business changes are processed by the same worker and broadcast to every port.

Why use this arrangement?

Federation lets A own and publish the client implementation without compiling that implementation directly into B and C's source. Separate projects can build their interfaces independently while agreeing on the exposed API. The worker centralizes mutation rules and serializes updates. RxJS gives Angular a stream of current snapshots instead of requiring components to poll or manually copy values between screens.

This is a frontend federation demonstration with a browser-memory store. It is not a deployed backend microservice. The development HTTP processes serve JavaScript, HTML, CSS and federation metadata. They do not store todos or receive counter-update API calls.

02 | Files, dependencies and running from scratch

The application-specific file map

Project-A/federation.config.js names the provider and exposes ./AppState. Project-A/src/app/app-state.service.ts contains the reusable page client. Project-A/public/state-bridge.html loads the relay script. Project-A/public/state-bridge.js connects that document to the worker. Project-A/public/state-worker.js contains the authoritative store and every business mutation.

Project-A/src/app/app.ts and app.html display the summary and include the Todo component. todo.ts and todo.html implement adding and completing tasks. Project-B/src/app/app.ts and app.html implement loading the remote client, editing and counter B. Project-C/src/app/app.ts and app.html implement loading the remote client, deleting and counter C. B/C app-state.ts files contain TypeScript contracts only.

The supporting files still matter to startup: package.json defines commands and dependencies; angular.json connects the federation builder, browser entry point and assets; src/main.ts initializes federation; src/bootstrap.ts creates Angular; app.config.ts registers global browser error listeners; src/index.html supplies the root element. CSS controls layout and completed-task styling, not synchronization.

Libraries used by this code

@angular/core provides Component, OnInit, signal, ApplicationConfig and provideBrowserGlobalErrorListeners. Component connects a class to a template and its imports. OnInit describes a lifecycle hook. signal holds B/C's client reference and loading error, causing template consumers to update when set(). These signals are not the common todo store.

@angular/common supplies AsyncPipe. The templates use it to subscribe to state$ and render each latest emission. @angular/forms supplies FormsModule in A and B, enabling ngModel and ngSubmit. C has no editable form, so it does not need this package in its application dependencies.

@angular/platform-browser supplies bootstrapApplication. @angular/compiler and @angular/compiler-cli support Angular template compilation; @angular/build and @angular/cli supply build and development tooling. TypeScript checks the typed source and is compiled to browser JavaScript. tslib supplies helpers used by compiled code when needed. These are infrastructure dependencies, not separate business services.

@angular-architects/native-federation supplies initFederation, loadRemoteModule and the build/configuration integration. es-module-shims supports the import-map loading path selected by the installed adapter. RxJS supplies Observable and shareReplay in page clients, and BehaviorSubject in the worker. The worker loads the RxJS UMD asset independently of the page's federated dependencies.

The inspected package.json files declare Angular packages ^22.2.0, federation ^22.2.2, RxJS ~7.8.0 and TypeScript ~6.0.2. These are version ranges, not proof that every installed patch equals the lower bound. package-lock.json records resolved installations. The explanations here use the installed adapter and RxJS source as well as the project code.

Browser APIs used directly

document.createElement and appendChild create the hidden bridge. window.postMessage and the message event communicate across parent/iframe origins. SharedWorker and MessagePort connect the bridges to one worker. structuredClone copies business state before mutation. crypto.randomUUID identifies a pending action. setTimeout and clearTimeout implement reply timeouts. Set tracks ports and allowed origins. pagehide and pageshow handle leaving and restoring the bridge document.

None of those browser APIs is an Angular service. In particular, a SharedWorker is different from a Service Worker: this code uses a shared computation/message context, not an offline request cache or network interceptor.

Start the existing repository

Open three terminals from the workspace and run the corresponding commands. npm ci installs the locked dependency tree; run it when dependencies are not installed or need a clean reproducible installation. Keep all three npm start processes running during the demo.

```
# Terminal 1
cd Project-A
npm ci
npm start

# Terminal 2, starting from the workspace root
cd Project-B
npm ci
npm start

# Terminal 3, starting from the workspace root
cd Project-C
npm ci
npm start
```

Open A at http://localhost:4200 and use its links to open B and C. Use the same Chrome or Edge browser profile and the exact localhost addresses. Do not mix localhost with 127.0.0.1: the client and bridge validate configured origins. The source has explicit development URLs; changing ports or deploying requires updating them consistently.

For production compilation, run npm run build separately inside each project. This creates static browser output; it does not create a database, persistent data service or authentication system. Serving the output still requires compatible origins, module loading headers, iframe policy and worker asset paths.

03 | Startup: from npm start to the first Angular view

Build preparation before the browser runs

npm start invokes ng serve from package.json. In angular.json, the serve target uses @angular-architects/native-federation:build and delegates to serve-original:development. The underlying Angular development server serves the application on its configured port: A 4200, B 4201, C 4202.

The esbuild target uses @angular/build:application, with browser set to src/main.ts. The federation build prepares exposed modules and dependency metadata in addition to normal Angular output. This is why remoteEntry.json exists at runtime even though it is not a handwritten source file under public.

A's assets configuration copies public files at stable URLs and copies node_modules/rxjs/dist/bundles/rxjs.umd.min.js to /worker/rxjs.umd.min.js. Without the second copy, the worker's importScripts call would fail even if federation successfully loaded the page client.

index.html: where Angular attaches

A's index.html contains <app-root>. B contains <app-counter-b>; C contains <app-counter-c>. Those tags match each root component's selector. Build tooling adds the generated script references. The index file is a document shell; it does not create the worker or load a todo list itself.

main.ts: initialize first, bootstrap second

Project-A/src/main.ts

```
1  import { initFederation } from '@angular-architects/native-federation';
2
3  initFederation()
4    .catch(err => console.error(err))
5    .then(() => import('./bootstrap'))
6    .catch(err => console.error(err));
```

The import makes initFederation available. Calling it starts asynchronous setup of the local federation environment. The installed adapter uses the local ./remoteEntry.json as host metadata and selects its shim import-map provider unless explicitly configured otherwise. This application supplies no remote manifest argument; B and C later provide A's URL directly when loading the exposed module.

The dynamic import('./bootstrap') delays evaluating Angular's bootstrap module until federation initialization settles. That boundary matters because imports in bootstrap.ts bring Angular and the root component into the runtime. The code intentionally creates the federation setup boundary before this dependency graph is evaluated.

Read the promise chain precisely. The first catch logs an initialization error and returns normally, so the following then still attempts to import bootstrap. It is not a hard stop. The final catch logs a rejected import or other rejection propagated through the chain. bootstrap.ts also has its own catch for bootstrapApplication. A logged failure does not guarantee a fully working page.

bootstrap.ts and app.config.ts

Project-A/src/bootstrap.ts

```
1  import { bootstrapApplication } from '@angular/platform-browser';
2  import { appConfig } from './app/app.config';
3  import { App } from './app/app';
4
5  bootstrapApplication(App, appConfig).catch((err) => console.error(err));
```

bootstrapApplication creates the root standalone application using App and appConfig. Angular instantiates the component, evaluates its template and renders its selector's element. The root component declares its own template dependencies in imports; there is no custom NgModule needed for these components.

Project-A/src/app/app.config.ts

```
1  import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
2
3  export const appConfig: ApplicationConfig = {
4    providers: [provideBrowserGlobalErrorListeners()],
5  };
```

provideBrowserGlobalErrorListeners registers Angular's global browser-error handling integration. It does not provide AppStateClient through dependency injection and does not establish federation or a worker. The exported appConfig is passed to bootstrapApplication.

A and B/C begin their state connection differently

A's App imports appState directly from its own app-state.service.ts. While that module graph evaluates, export const appState = new AppStateClient() constructs the client and starts creating the bridge. Therefore A's connection can begin before its first template subscription. The client retains the latest view so a later subscription still sees the current value.

B and C initially have appState = signal(null). Angular starts their root components, then invokes ngOnInit. Their hook awaits loadRemoteModule for A's exposed client. Evaluating that remote module creates a client in the consuming page. Setting the signal exposes it to the template, which can then subscribe to state$.

There are consequently two visible waiting stages in B/C: loading the federated module and connecting that loaded client to the store. These stages can fail independently and have different error messages.

04 | Native Federation: configuration and remote URL resolution

The exact provider configuration

Project-A/federation.config.js

```
const { withNativeFederation, shareAll } = require('@angular-architects/native-federation/config');

module.exports = withNativeFederation({
  name: 'project-a',
  features: {
    ignoreUnusedDeps: false,
  },
  exposes: {
    './AppState': './src/app/app-state.service.ts',
  },
  shared: {
    ...shareAll({ singleton: true, strictVersion: true, requiredVersion: 'auto' }),
  },

  skip: [
    '@angular/platform-browser/animations',
    '@angular/platform-browser/animations/async',
    'rxjs/ajax',
    'rxjs/fetch',
    'rxjs/testing',
    'rxjs/webSocket',
  ],

});
```

withNativeFederation produces configuration consumed by the federation builder. name identifies the project as project-a in generated metadata. exposes maps a public lookup key, ./AppState, to the source file to compile. The public key is not a filesystem request to /AppState.ts; it is an identifier that the runtime resolves through generated metadata.

shareAll builds a shared-dependency configuration from package dependencies. singleton: true requests one resolved dependency instance for that package within the federation environment of a page. strictVersion: true requests strict version compatibility. requiredVersion: 'auto' derives requirements from package metadata. These options concern dependency resolution; they do not serialize application data or share an object across browser tabs.

features.ignoreUnusedDeps: false keeps automatic unused-dependency filtering disabled. In this project the Angular bootstrap sits behind a dynamic import, so retaining the configured dependency set is intentional. The skip array excludes the listed animation and RxJS secondary entry points from federation sharing. It does not remove the main rxjs package used by the client.

B and C have their own project names and shared dependency configuration. They do not expose counter components in the current implementation. A does not download and embed B/C's UI. The current federation direction is B -> A and C -> A for the client module.

The consumer's call

```
const module = await loadRemoteModule({
  remoteEntry: 'http://localhost:4200/remoteEntry.json',
  exposedModule: './AppState',
});
this.appState.set(module.appState);
```

remoteEntry is the absolute HTTP address of A's metadata. exposedModule must match A's exposes key exactly, including spelling and case. module is the loaded JavaScript module namespace. module.appState is

its named export; it is not a server response containing current todos.

What the installed adapter does

The installed file @angular-architects/native-federation/src/index.js normalizes the call options and awaits its stored federation initialization promise. Because the application passes remoteEntry but no remoteName, the adapter fetches the metadata and reads its name. A non-OK response or missing name causes an error.

It then invokes federation.initRemoteEntry(remoteEntry, remoteName) to register that remote and its dependency information. Finally it delegates to federation.loadRemoteModule(remoteName, exposedModule). The underlying federation/import-map machinery resolves the exposure to its generated JavaScript asset and integrates shared imports before evaluating the requested module.

The code path can involve metadata discovery and initialization reads, and browser caching can change what Network displays. Do not claim that every load always makes exactly one HTTP request. The important chain is metadata -> remote identity and exposure mapping -> resolved module and dependencies -> evaluated named export.

```
Illustrative metadata shape, not a fixed generated filename:
{
  "name": "project-a",
  "exposes": [
    { "key": "./AppState", "outFileName": "AppState-<hash>.js" }
  ],
  "shared": [ ... dependency metadata ... ]
}
```

The build decides the actual filenames. The browser obtains the module asset from A's deployment location using the metadata. B and C should not hard-code a generated hash or edit remoteEntry.json manually. The stable contract is A's entry URL, exposed key and exported API.

Import maps and the execution environment

JavaScript source can import a package name such as rxjs. An import map connects such module specifiers to browser-loadable URLs. Federation manages those mappings and dependency compatibility. The installed Angular adapter selects es-module-shims integration by default; angular.json includes es-module-shims in polyfills.

Downloaded code executes in its consumer's JavaScript environment. Loading A's module into B does not navigate B to A or change B's window origin. document inside the loaded client is B's document. Its hidden iframe is therefore appended to B's page, with its src explicitly pointing to A.

What federation does not carry

Changing a counter does not fetch remoteEntry.json again as part of the business action. Metadata and module loading establish access to the client code. Subsequent actions use postMessage and MessagePort. There is no HttpClient request, REST endpoint or WebSocket for the todo/counter business state in the custom implementation.

Federation can distribute components as well as non-visual modules in general. Here the exposure is a state client. Describe this demo accurately as three independently running Angular frontends using a federated client and one shared browser store, rather than claiming A renders two remote components.

05 | The data contract and message protocol

Business state versus local view state

```
// Authoritative data held by the worker
{ countB: 0, countC: 0, todos: [] }

// Example task in todos
{ id: 1, text: 'Prepare demo', completed: false }

// Client adds status fields for this page
{ countB: 0, countC: 0, todos: [],
  connected: false, pending: false, error: '' }
```

The worker never stores connected, pending or error. Those fields describe a particular client's relationship to the worker. One page can be disconnected or waiting while another is successfully showing the shared data. TodoItem.id is allocated centrally; text is validated and trimmed; completed is a boolean.

StateAction is a TypeScript union describing five command shapes. counter includes project b/c and increment/decrement/reset. addTodo includes text. toggleTodo and deleteTodo include id. editTodo includes id and text. This improves correctness while compiling TypeScript, but interfaces and unions are erased at runtime. The worker still validates incoming commands.

Messages, sender, receiver and effect

connect travels from page to bridge with channel project-a-state. The bridge uses it to start a worker connection. action travels from page through bridge to worker, carrying a requestId and an action object. state travels from worker to every bridge and then every page, carrying a complete business snapshot.

done travels only to the requesting port, then its page, with the matching requestId. It means applyAction completed without throwing. error also goes to the requester when an action is rejected. The bridge can additionally send a fatal error for a worker connection problem. disconnect travels from a departing bridge to the worker so its port can be removed.

```
// Example outgoing page message; UUID shortened for readability
{
  channel: 'project-a-state', type: 'action', requestId: 'req-B-1',
  action: { type: 'counter', project: 'b', action: 'increment' }
}

// Worker broadcast, before bridge adds channel
{ type: 'state', state: { countB: 1, countC: 0, todos: [] } }

// Worker completion reply only to B's port
{ type: 'done', requestId: 'req-B-1' }
```

The page/iframe boundary uses a channel label and origin/source checks. The worker connection uses a MessagePort; the worker reads the type and requestId rather than relying on a window origin property. The bridge forwards the action message intact and adds channel to replies sent back to its parent.

Why a snapshot and an acknowledgement are separate

Suppose B submits an edit while C increments a counter. B can receive the counter's state snapshot before its own edit is processed. If any state message cleared B's pending flag, B would believe its edit had finished prematurely. This implementation keeps pending unchanged on state and clears it only for a matching done/error, or a fatal failure/timeout.

Messages carry structured-cloned values across these boundaries. A, B and C do not receive one shared JavaScript object reference. They each receive copies describing the same authoritative state. Methods and subscriptions stay in their own execution contexts; the plain data travels.

06 | AppStateClient: every field and function

File: Project-A/src/app/app-state.service.ts. Despite the filename, this is an ordinary TypeScript class with one exported instance per evaluated module environment. It has no @Injectable decorator and is not injected through Angular's provider system.

Fields and construction

view starts with zero counters, an empty list, connected false, pending false and no error. It remembers the latest received values even before an observer exists. emit is an optional callback into the observable's source subscriber. bridge refers to this client's iframe. replyTimeout holds the timer handle. pendingRequest holds the currently outstanding UUID or undefined.

The constants projectAOrigin and channel are used in every parent/iframe exchange. The URL is intentionally absolute because the same source runs on B and C. A relative bridge URL in B would request B's origin, losing the common A-origin worker context.

The constructor creates a hidden iframe, gives it an accessible title and sets src to A's /state-bridge.html. It installs an onload callback that posts connect to the iframe using the exact target origin. It registers this.receive as the page message listener before appending the iframe, then starts the reply timer. Appending starts loading the bridge document. The constructor does not synchronously obtain the store.

The final export, new AppStateClient(), executes during module evaluation. It starts a page-lifetime connection. Merely reading a TypeScript interface cannot do this; evaluating the exported runtime instance can.

state$: from incoming browser messages to observable emissions

```
readonly state$ = new Observable<AppState>(subscriber => {
  this.emit = view => subscriber.next(view);
  subscriber.next(this.view);
  return () => { this.emit = undefined; };
}).pipe(shareReplay({ bufferSize: 1, refCount: false }));
```

Constructing Observable defines a subscription function; that function runs when the source is subscribed to. It stores a callback that sends later views to the source subscriber, then immediately sends the current view. The returned function is teardown for the source subscription. The dollar sign in state$ is a convention, not language syntax with special behavior.

shareReplay creates a shared observable around that source. The first template subscription starts one source subscription. Additional local template subscriptions share it and receive the latest buffered value. This is essential here because the source only stores one emit callback: multiple independent subscriptions to the unshared source would overwrite that callback.

bufferSize: 1 keeps the most recent emission. refCount: false means that, once started, the source is not disconnected merely because all current downstream subscribers unsubscribe. The design treats the client as living for the page lifetime. AsyncPipe still unsubscribes its own downstream subscription on destruction; that does not automatically destroy the client or iframe.

If a worker reply arrives before the first template subscribes, publish stores it in view and emit is absent. When the source is eventually subscribed, subscriber.next(this.view) delivers that stored snapshot. If A's root and Todo subscribe at different times, replay gives the later subscriber the latest value without waiting for another user action.

publish(view)

This method assigns this.view and invokes this.emit?.(view). Assignment updates the cache; the optional call emits only if a source subscriber has registered the callback. It does not post to the worker, mutate todos or confirm an action. All UI-state transitions use it so the cache and rendered stream agree.

updateCounter(project, action)

Builds {type: 'counter', project, action} and returns send(...)'s boolean. B supplies project b; C supplies c. The action distinguishes increment, decrement and reset. The client does not calculate the new number, avoiding stale read-modify-write calculations in individual tabs.

addTodo(text), toggleTodo(id), editTodo(id, text), deleteTodo(id)

Each method constructs its matching StateAction and delegates to send. addTodo supplies raw input text; the worker ultimately trims and validates it. toggleTodo sends only an ID, allowing the worker to invert the current completion state. editTodo supplies an ID and proposed text. deleteTodo supplies the ID to remove. None changes view.todos directly.

send(action): admission, tracking and delivery

The first guard returns false if disconnected or already pending. This client allows one outstanding action. Other clients have independent pending flags and may submit concurrently. If admitted, crypto.randomUUID creates pendingRequest; publish sets pending true and clears the visible error; waitForReply resets the timeout; postMessage sends the command to this bridge; the method returns true.

A true return means accepted for sending, not persisted, validated or committed. postMessage does not synchronously wait for the worker. The constructor's iframe must load, the bridge must process the action and the worker must reply. Angular buttons are disabled using pending, and the send guard also protects callers that bypass the template.

Because the send result is immediate, A clears its input and B clears its edit form before the later done response. If the worker rejects the command, the current code displays the error but does not restore the cleared draft automatically. This is a current behavior, not a guarantee of successful saving.

receive(event): trust boundary and three branches

receive is an arrow-function field, so this remains bound to AppStateClient when the browser calls it as an event listener. It first checks event.origin equals projectAOrigin and event.source equals this bridge's contentWindow. Origin alone would not distinguish another A-origin window. It then checks the channel. Unrelated messages return without affecting the client.

For a valid state message it validates business fields, merges the snapshot into a view with connected true, preserves this client's pending flag and clears error. If there is no pendingRequest, it clears the initialization/reply timer. If an action is still outstanding, a snapshot does not stop that action's timeout.

For done, the requestId must equal pendingRequest. It clears the timer, clears pendingRequest and publishes pending false while retaining data. For error, either fatal must be truthy or its requestId must match. A string message is displayed, otherwise a default message is used. showError handles the actual state transition.

Unknown message types, mismatched request IDs and malformed state messages are ignored. An ignored initialization response can leave the client waiting until timeout. There is no retry queue or request-history log.

validState(state)

Checks safe integer countB/countC; confirms todos is an array; checks every task has a safe integer ID, string text and boolean completed. It validates data from a message boundary before rendering. It does not check local status fields because the worker does not send them. It also does not enforce unique IDs or nonblank text here; the current worker produces those properties through its mutation rules.

showError(error, disconnected)

Clears the timer and pendingRequest, publishes pending false and the message, and sets connected false only when disconnected is true. A rejected task edit can therefore leave the connection usable. A fatal bridge failure or timeout marks it unusable in the local UI. Existing data remains visible in the cached view, although counter templates show a dash when disconnected.

waitForReply()

Cancels the previous timeout and starts a 10,000 millisecond timer. Expiry calls showError with a reload/start-A message and disconnected true. It is a deadline for initialization or the current action, not a heartbeat that continuously checks idle connections. Timing out does not cancel an already posted action in the worker.

If a valid state arrives later, the state branch can mark the client connected again. A late done whose ID no longer matches the cleared pendingRequest is ignored. There is no automatic idempotent retry; the user should inspect the current state before repeating an uncertain operation.

07 | RxJS internals in this application

Observable, observer, subscription and Subject

An Observable describes a source of values over time. A subscriber receives notifications, usually next(value), and potentially error or complete. A subscription represents that connection and supports unsubscribe. In the page client the producer is browser-message handling calling publish, which invokes subscriber.next through emit. No polling interval is required.

A Subject can receive values and multicast them to subscribers. A BehaviorSubject additionally requires an initial value and remembers its current value. The worker needs a current snapshot immediately, so BehaviorSubject is appropriate: a new worker starts with zero counters and an empty array, and applyAction can read store.value before computing the next snapshot.

The installed RxJS BehaviorSubject implementation assigns its internal current value before notifying subscribers in next(value). Its subscription path sends the current value to a newly registered subscriber. In this worker, however, there is one broadcast subscription established at worker startup, not one RxJS subscription per browser tab.

The worker's actual RxJS sequence

```
const store = new rxjs.BehaviorSubject({ countB: 0, countC: 0, todos: [] });
store.subscribe(state => {
  for (const client of clients) client.postMessage({ type: 'state', state });
});
// Later, after a validated action:
store.next(nextState);
```

The abbreviated sample above shows the sequence; the actual source wraps each port post in try/catch. The subscription initially receives the empty state, but clients is empty then. A newly connected port later receives store.value explicitly in self.onconnect. This explicit send is what gives a late-joining tab its snapshot; it is not a new BehaviorSubject subscription for that tab.

For a valid mutation, store.next replaces the current value and synchronously invokes the broadcast subscriber. The subscriber posts asynchronous messages to the ports. RxJS notification inside the worker is synchronous; delivery into another browser context is asynchronous. The worker finishes enqueuing broadcasts before posting done to the requester.

The client-side replay layer

shareReplay in the page uses a ReplaySubject internally in the installed RxJS implementation. Therefore 'one common RxJS store' does not mean there is literally only one RxJS object anywhere. It means one authoritative business-state BehaviorSubject. The page's replay subject and view hold received snapshots for rendering; they do not accept independent business mutations.

This matters when explaining singleton: dependency sharing, observable multicast, replay caching and shared-worker state ownership are four different concepts. They cooperate, but none should be used as a synonym for the others.

AsyncPipe connects the stream to Angular

The expression state$ | async subscribes to the observable, returns the latest received value to the template and marks the view for checking on updates. Angular then evaluates interpolations, disabled bindings and the @for list against that snapshot. The pipe unsubscribes when its view is destroyed or its observable input changes. The local shareReplay configuration determines whether the source itself stays subscribed.

No component manually subscribes with .subscribe in its class. This keeps normal template subscription cleanup with AsyncPipe. The worker's explicit store.subscribe is different: it exists for the worker lifetime to broadcast every committed snapshot.

Why error messages are values instead of observable errors

The application publishes error: 'message' in AppState. It does not call subscriber.error when a task is rejected. An RxJS error notification would terminate that subscription; a UI error field allows the observable to continue delivering later successful snapshots. Similarly, done changes pending through another next emission rather than completing the stream.

Why no map, merge, debounceTime or switchMap?

The current custom code only imports Observable and shareReplay in the client and creates BehaviorSubject in the worker. It forwards whole snapshots and handles one local request at a time. No search request cancellation, debounced text saving or transformed selector pipeline is implemented. Do not claim these operators are being used just because they are common RxJS patterns.

08 | The A-origin bridge: every callback

Why an iframe is necessary here

Origin consists of scheme, hostname and port. The three localhost ports are different origins. A script downloaded from A and evaluated in B still belongs to B's document environment. The client therefore creates an actual document at A's origin inside each page. All bridge documents use /state-worker.js from that same origin and the same worker name.

In the supported same-browser-profile localhost setup this gives them access to one common worker. General deployments also depend on browser partitioning and iframe/worker policies; merely using a common filename on unrelated origins does not create a global worker shared across the internet.

state-bridge.html

Project-A/public/state-bridge.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Project A shared store bridge</title>
</head>
<body>
  <script src="state-bridge.js"></script>
</body>
</html>
```

The HTML loads state-bridge.js as a normal script. It does not bootstrap Angular, display project features or contain a store. Keeping it in public gives it a stable URL independent of the generated federation module hash.

Top-level state

channel is project-a-state. allowedOrigins is a Set containing the three exact development origins. parentOrigin starts undefined and is recorded only after a message passes validation. worker stores the SharedWorker connection object. The variable prevents duplicate connections from repeated connect messages while the connection exists.

sendToParent(message)

If parentOrigin is known, it calls window.parent.postMessage with {channel, ...message} and the exact parentOrigin. That adds the protocol label to worker replies and sends them to the owning application. Without a known validated parent, it sends nothing. It forwards state, done and error; it does not interpret todo data.

connectionError()

Creates a fatal error message explaining that the shared store connection failed and suggesting Chrome or Edge and reload. It is attached to both worker.onerror and worker.port.onmessageerror. A synchronous failure while constructing the worker also calls it through catch. Its job is to turn a transport/startup failure into a page-visible status.

connect()

Constructs new SharedWorker('/state-worker.js', 'project-a-rxjs-store'). The URL resolves against the bridge's A origin. The name identifies the intended shared instance together with the worker's origin/script and browser context. The returned object provides a port to the shared worker; constructing another connection does not necessarily run a new independent store.

It attaches error handlers, sets port.onmessage to forward event.data through sendToParent, and calls port.start. Assigning onmessage already starts a port in standard behavior; the explicit start here makes activation clear and is harmless. Any synchronous construction error becomes connectionError.

window message listener

It rejects messages unless event.source is window.parent and event.origin belongs to allowedOrigins. It rejects the wrong channel. After validation it stores parentOrigin. A connect message calls connect only if worker is absent. An action message is forwarded with worker.port.postMessage only if worker exists. The bridge does not queue actions before connecting; the client normally prevents them using connected false.

pagehide listener

If connected, it posts disconnect, closes its port and clears worker. The worker removes that port and may terminate if it was the last one. This implements intentional cleanup when the iframe/page leaves. Abrupt process termination can bypass orderly lifecycle events; the code is not a durable session registry.

pageshow listener

When a document is restored from the browser's back/forward cache, event.persisted is true. If parentOrigin is known and worker is absent, it reconnects. The worker then sends its current snapshot. A restore is different from evaluating a fresh document, so this handler covers the cached-page path.

What the bridge deliberately does not do

It has no BehaviorSubject, no copy of mutation logic, no localStorage access and no HTTP calls for updates. It is a relay and origin boundary. Removing it while keeping separate ports would remove the common same-origin access path used by this design.

09 | SharedWorker: every function and business rule

Startup and owned variables

importScripts('/worker/rxjs.umd.min.js') synchronously loads the classic RxJS bundle and exposes rxjs in the worker global environment. The worker has no document or page template. The RxJS code here is a separate runtime instance from RxJS loaded by federation in a page.

store is initialized once when this worker starts. clients is a Set of connected MessagePorts. nextTodoId begins at 1 and increments for created tasks. IDs belong to the common authority, so two tabs do not allocate them from separate counters. Deleting a task does not decrement the sequence; restarting the worker resets it.

store.subscribe(state => ...)

Loops through clients and posts {type: 'state', state} to each port. If posting throws synchronously, it removes that port from the set and continues. A successful post means queued for delivery, not proof that a page rendered the value. There is no per-recipient delivery acknowledgement for snapshots.

taskText(text)

Verifies that text is a string and text.trim() is nonempty. Otherwise it throws 'Task text cannot be blank.' It returns the trimmed string. addTodo and editTodo use it, so validation remains centralized even if a caller bypasses disabled UI buttons.

Example: ' Prepare demo ' becomes 'Prepare demo'. A string containing only spaces is rejected. The code does not impose a maximum length or perform text searching. Angular interpolation displays task text as text; it is not inserted using an innerHTML binding.

findTask(state, id)

Finds the task whose id strictly equals the supplied ID. If absent it throws 'This task no longer exists. Use the updated list.' It returns the actual task object from the supplied state's array. Because applyAction supplies a cloned state, editing this returned task changes the copy, not store.value before commit.

This check is important when C deletes a task while B has its editor open. A stale ID must not silently create a new task or edit an arbitrary array index. Stable IDs identify tasks even when deletion changes array positions.

updateCounter(state, action)

Validates project is b or c and action is increment, decrement or reset. It chooses countB or countC with a conditional expression, changes that property in the cloned state, then verifies Number.isSafeInteger. Negative values are allowed; the code has no minimum-zero guard. Reset always sets the selected counter to zero. An invalid or unsafe result throws before commit.

This helper returns no separate state. It mutates the state copy passed by applyAction. That is safe within the current synchronous function because only store.next publishes the completed copy.

applyAction(action): copy, choose, mutate, publish

First rejects a missing action. Then structuredClone(store.value) creates a separate nested copy, including todo objects. A shallow object spread would leave nested todos shared unless separately copied. The deep copy lets the code use simple property assignments while keeping the old store snapshot unchanged until successful completion.

The switch handles counter by calling updateCounter. addTodo validates text, appends {id: nextTodoId++, text, completed: false} and breaks. toggleTodo finds the current task and negates its completed flag. editTodo finds the task and replaces only text after validation. deleteTodo first confirms existence, then filters out that ID. The default throws 'Unknown action.'

After a successful branch, store.next(state) is called exactly once. If validation throws before this line, the original business state remains current. This copy-before-commit design is not a database transaction or durable log, but it prevents partially applying the ordinary validation failures handled here.

For reset of an already-zero counter or an edit that keeps the same text, the current code still creates a copy and publishes it. There is no equality check or distinctUntilChanged selector suppressing such emissions. Every connected page receives the full state, including fields it may not display.

self.onconnect(event)

When a bridge connects, event.ports[0] provides its worker-side port. The worker adds it to clients, assigns the message handler, starts the port and explicitly posts store.value as an initial state. Existing clients are not reinitialized when a new client joins.

port.onmessage(event): disconnect branch

For disconnect it removes the port, closes it and calls self.close if clients.size is zero, then returns. This is why closing the last connected application normally discards the memory state. Closing only A's visible tab is different: B and C can keep their A-origin bridges and worker connections alive.

port.onmessage(event): action branch

It ignores messages that are not action or lack a string requestId. For an accepted envelope, try calls applyAction. On success it posts done with the same ID to this port. On a thrown validation error it posts error with that ID and error.message. Other ports receive successful state changes, but not this request-specific done/error.

The worker does not keep previously processed request IDs. A requestId correlates a reply; it is not a deduplication key in this implementation. Manually replaying the same increment message could increment again. No automatic retry is implemented, so the normal UI does not intentionally replay commands.

Sequential processing and its limits

Worker message callbacks execute one at a time. applyAction contains no await or delayed mutation. Each accepted action therefore reads the latest store.value, commits and returns before another message callback applies its change. This prevents the usual lost increment caused by two page-local calculations from the same old value.

Sequential handling does not make every conflict disappear. Two edits to the same task are applied in arrival order and the later applied text wins. A delete before an edit causes the edit to fail. There are no revisions, user locks, conflict prompts or distributed ordering across devices.

10 | Angular feature code: every project and template

Project A root: App

App imports AsyncPipe, Component, appState and Todo. Component's selector is app-root; templateUrl and styleUrl connect the view and styles. imports [AsyncPipe, Todo] makes the pipe and child component usable in its standalone template. The class exposes state$ = appState.state$ and has no mutation methods.

app.html uses an @if alias for the latest observable view. It displays countB and countC when connected and a dash otherwise. It shows state.error or a connecting message. The child <app-todo /> handles the tasks. The B/C anchors use target _blank and rel noopener; their links open applications without passing state through window.opener or query strings.

Project A Todo: fields and methods

Todo imports FormsModule and AsyncPipe. newTask is the local input string. state$ points at the same client observable used by A's root, which is why local replay/multicast matters. There is no second worker connection created by the Todo component importing the same evaluated module.

addTask calls appState.addTodo(this.newTask). If the client admits the send, it clears newTask. If disconnected/pending, send returns false and the input stays. toggleTask(id) calls appState.toggleTodo(id); it does not invert a local todo object.

todo.html binds [(ngModel)] to newTask and (ngSubmit) to addTask. The input's name lets Angular register it with the form. Add is disabled for blank trimmed text, disconnected state or pending. @for tracks task.id so list identity survives text edits and unrelated updates. [checked] reads completed; (change) sends a toggle. [class.completed] applies visual styling. @empty distinguishes loading from a genuinely empty connected list.

Project B: loading and local state

App implements OnInit. appState is a signal whose value starts null; error is another signal for loading failures. editingId and draftText are ordinary local component fields. The import type of AppStateService and TodoItem is erased from runtime JavaScript; it supplies compile-time descriptions.

ngOnInit awaits A's module and sets module.appState into the signal. If remote loading rejects, it sets the load error. This catch concerns federation/module loading. A later worker failure appears in service.state$ as state.error, not in this separate signal.

increment, decrement and reset call updateCounter('b', correspondingAction) using optional chaining. If appState is still null, no call happens. The template normally hides controls until the client exists and disables them until connected and not pending.

startEditing(task) saves task.id and copies task.text into draftText. It sends no shared action. Typing changes only the local draft. saveTask(id) trims the draft; if nonblank and editTodo returns true, it calls cancelEditing. cancelEditing sets editingId to null and draftText to empty; it has no worker side effect. Cancel therefore never changes A or C.

Project B template: two asynchronous layers

@if (appState(); as service) first waits for the remote module. Inside, service.state$ | async obtains current connection and data values. The counter reads countB. Each task row either displays text with an Edit button or a form for the matching editingId. Save submits saveTask(task.id); Cancel calls cancelEditing and is type button so it does not submit the form.

The edit input uses a task-specific id and matching label, while ngModel writes draftText. A shared state update can change the task list without replacing the draft. If another B tab edits the same task while this editor is open, the draft can become stale; saving sends the user's current draft, with no revision comparison.

Project C: methods and template

C's ngOnInit follows B's remote loading sequence. Its three counter methods select project c. deleteTask(id) delegates to deleteTodo. There is no confirmation dialog or undo in this source. The template subscribes to the loaded client's state$, displays countC, lists shared tasks and sends a deletion by stable ID.

C displays completed styling but cannot toggle completion through its controls. It has no form input and does not import FormsModule. It uses the same loading-error versus state-error distinction as B. All counters and delete buttons are disabled while that client is disconnected or pending.

B/C app-state.ts: contracts, not stores

Both define TodoItem, AppState and AppStateService. AppStateService describes the observable plus the methods needed by that project: counter/edit for B and counter/delete for C. It creates no object and holds no data. The remote runtime object actually implements the fuller API owned by A.

These locally maintained interfaces do not prove at runtime that a newly deployed remote still has those methods. A compatible API is required. They are also not access controls: narrowing the TypeScript interface does not prevent browser code from calling other exported methods. The separation is intentionally at the UI level.

11 | Scenario: add, edit, complete and delete one task

Initial condition

Start one worker with countB 0, countC 0, todos empty and nextTodoId 1. Connect A, B and C. Each receives the snapshot and publishes connected true, pending false. A's form is available, B/C show an empty-task message and all counters show zero.

A adds 'Prepare demo': complete call chain

1. The user types ' Prepare demo '. ngModel changes Todo.newTask only. No shared message has been sent yet.

2. Form submission calls Todo.addTask. That calls appState.addTodo(newTask), which builds the addTodo action and enters send.

3. send checks connection/pending, allocates UUID rA, publishes pending true and starts the 10-second deadline. The form's Add and checkbox controls become disabled for A. B/C remain independently usable.

4. The client posts channel/action/requestId to its own iframe at A's origin. send returns true, so Todo clears newTask. This happens before the worker commits.

5. The bridge validates parent source, origin and channel, then forwards the envelope through its worker port. The worker receives it in that port's onmessage callback.

6. applyAction clones the current store. taskText trims the text. The addTodo case appends {id: 1, text: 'Prepare demo', completed: false}; nextTodoId advances to 2.

7. store.next publishes the new snapshot. The broadcast subscription queues a state message on every connected port. After applyAction returns, the worker queues done/rA only on A's port.

8. Each bridge forwards its state to its parent. Each client's receive validates it and publishes the new view. A keeps pending true until its done; B/C keep their own pending values.

9. AsyncPipe receives each emission and Angular renders task 1 in all lists. A's checkbox is unchecked; B offers Edit; C offers Delete.

10. A receives the matching done, clears its timer and pendingRequest, and publishes pending false. Its controls become available again. Rendering can occur after queued processing; do not assume all three tabs repaint at the exact same instant.

B edits to 'Present demo'

Clicking Edit calls startEditing(task). editingId becomes 1 and draftText becomes 'Prepare demo'. Typing 'Present demo' still affects B only. Cancel would discard this local draft without an action.

Save calls saveTask(1), trims the draft and calls editTodo(1, 'Present demo'). The same send/bridge/worker chain runs. findTask finds ID 1 in the cloned latest store; taskText validates the proposed text; only text is replaced. ID 1 and completed false remain unchanged.

The worker emits the updated snapshot to all tabs. A and C now render 'Present demo'. B's form had already closed when send returned true; done clears its pending state. This is why editing is not implemented by directly modifying the task object inside B's received array.

A completes and reopens the task

Checkbox change calls toggleTask(1), then toggleTodo(1). The worker reads the latest completed value and applies !task.completed. The committed task becomes completed true and all templates apply completed styling. A second accepted toggle changes it back to false. Text and ID stay unchanged.

C deletes the task

Delete calls deleteTask(1), then deleteTodo(1). The worker checks existence and filters todos to remove ID 1. store.next broadcasts an empty array. All three @for blocks now select @empty. done clears C's pending flag. nextTodoId remains 2, so a subsequent new task gets ID 2 during this worker lifetime.

State progression to show in a demo

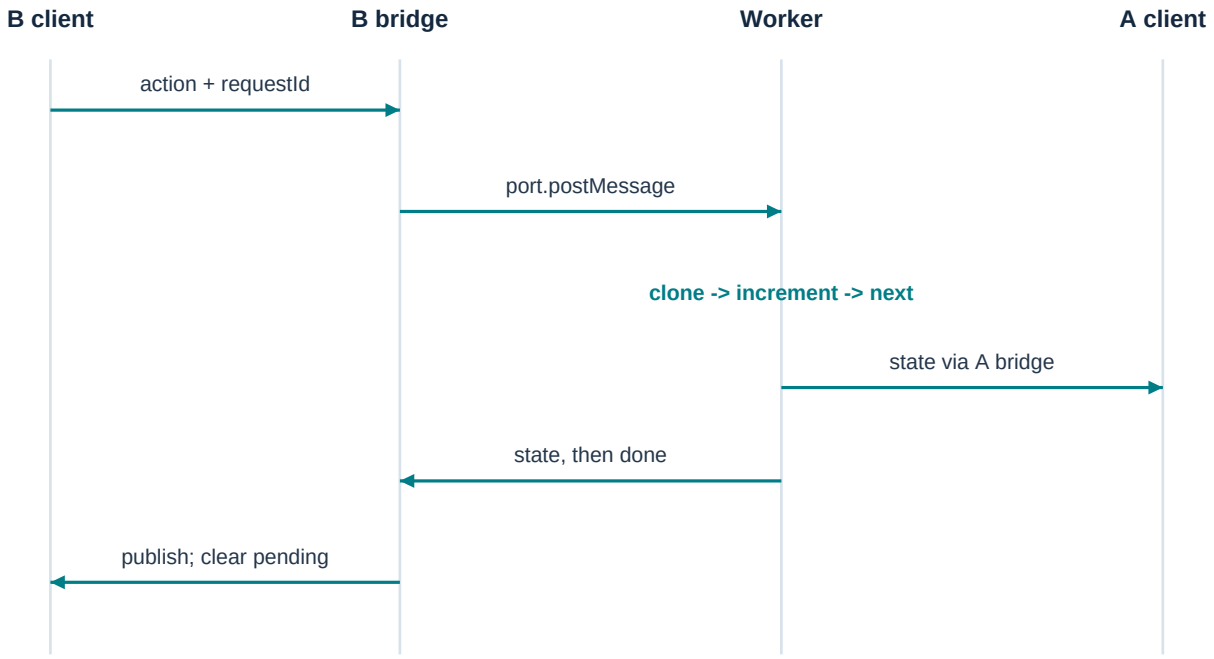
```
Start:    { countB: 0, countC: 0, todos: [] }
Add:      todos = [{ id: 1, text: 'Prepare demo', completed: false }]
Edit:     todos = [{ id: 1, text: 'Present demo', completed: false }]
Complete: todos = [{ id: 1, text: 'Present demo', completed: true }]
Delete:   todos = []
```

The counter values are preserved through all four todo mutations. Every mutation begins from the entire latest store snapshot, changes only the requested part and publishes the full result.

12 | Scenario: counters, simultaneous tabs and acknowledgement order

B increments while A displays the summary

Assume countB is 5 and countC is 2. B's + button invokes increment, which sends counter/b/increment. The worker clones the current state, chooses countB and increments it to 6. countC remains 2 and todos remain unchanged. All clients receive {countB: 6, countC: 2, todos: ...}. A renders both values, B renders 6, and C still renders 2.



No page sends the proposed number 6. It sends the operation increment. That lets the single authority calculate against its latest state. C decrement uses the same route with project c; reset changes only the selected counter. Values below zero are valid in this demo.

Two B tabs increment nearly together

Assume two B tabs display countB 5. Each sends an increment with its own UUID and each sets its local pending flag. Whichever port's message is handled first clones 5 and commits 6. The next callback clones 6 and commits 7. Both tabs and A eventually display 7. They can observe intermediate snapshots depending on rendering timing.

If each tab instead sent its locally calculated replacement value 6, the second write could overwrite the first with 6. This code avoids that lost-update pattern by sending operations and keeping synchronous read-modify-commit in one worker.

A broadcast does not finish someone else's action

Suppose B is pending on request rB and C is pending on rC. B receives C's committed snapshot, but receive preserves B's pending flag. Only done/rB or error/rB resolves B's action. A done/rC message would be ignored by B even if received, because IDs do not match. In ordinary operation it is not sent to B's port at all.

Ordering inside and across contexts

Within applyAction, store.next synchronously triggers port.postMessage for the snapshot. The worker then queues done on the requesting port. This establishes state-before-done for that request on that channel. Other tabs receive their snapshots asynchronously and may render at different times; there is no global repaint barrier.

The connected application is eventually updated through messages during normal operation. This is not a networked distributed database guarantee. No durable log, retransmission, version vector or transaction across separate worker instances exists.

Conflict: two editors modify the same task

B1 and B2 both start with 'Prepare demo'. B1 saves 'Present demo'; B2 later saves 'Review demo'. Each edit is valid for the existing ID, so the last processed edit wins. There is no stale-version rejection. A local draft is intentionally not overwritten by every broadcast, which preserves typing but allows stale edits.

Conflict: delete arrives before edit

C deletes ID 1 while B has a draft open. If delete commits first, findTask in the subsequent edit throws. The worker sends an error only to B and does not emit an edit snapshot. B shows the error and remains connected. Its editor may already have closed because saveTask clears it when send returns true.

If the edit commits first, it is briefly visible, then the delete removes the task. Both outcomes follow the worker's arrival order. This is a useful scenario for explaining what the code guarantees and what it does not.

13 | Scenario: late join, reload, closing A and failures

Open B after data already exists

B first downloads its own app, initializes federation and loads A's client. Its bridge connects to the existing worker. self.onconnect adds the new port and immediately sends store.value. B receives the current tasks and counters without requiring another edit. No browser storage restore is involved.

Reload A while B or C remains connected

A's old bridge disconnects. Other ports keep clients.size above zero. A's new document evaluates a fresh client and reconnects. The worker sends its current state, so A recovers the data from the still-running worker. Its local unsent input is lost because that belonged to the old page.

Close A's visible tab

B/C can remain connected because their hidden bridges are A-origin documents in their own tabs. The common worker is not stored inside A's root component. A's development server process should remain running so newly opened tabs can fetch the client, bridge and worker assets. Closing a tab and stopping the HTTP process are different operations.

Already loaded pages may keep exchanging worker messages even if an asset server stops, but new loads/reloads can fail. Do not use that as an offline-support claim: this application has no deliberate offline installation/cache strategy.

Close every connected application

Each normal pagehide sends disconnect. When the worker observes the last registered client removed, it calls self.close. A newly created worker starts with zeros, an empty list and nextTodoId 1. A browser crash or worker termination also loses the memory state. Reloading the sole connected tab can reset state because it creates a last-disconnect gap.

A's remote entry cannot be loaded

B/C ngOnInit catches loadRemoteModule rejection and sets the loading-error signal. Check A's process, the exact URL, HTTP status, generated metadata and shared dependency compatibility. No state client may have been created yet, so changing worker logic would not address a federation-load failure.

Remote loads but worker cannot start

The client iframe can load while /state-worker.js or /worker/rxjs.umd.min.js fails, or a browser policy can prevent worker creation. connectionError sends a fatal error through the bridge. The client clears pending and marks connected false. If no explicit error reaches the page, its 10-second timer eventually reports no reply.

Wrong origin or unrelated message

Messages with a disallowed origin, wrong source window or wrong channel are ignored. A hostname/port mismatch can look like a connection timeout. This validation is necessary for the relay, but it is not application user authorization. The worker's action API has no account identity or role checks.

A command times out

The client clears pending and marks disconnected. The worker may already have processed the action even if its reply was delayed or lost. Timeout reports uncertainty about communication; it is not a rollback instruction. There is no cancel message, durable request record or deduplication cache. Reconnect/reload and inspect the current state before manually repeating an uncertain action.

Limitations relevant to the demo

Data is local to the browser's worker-sharing context, not shared between computers or browser profiles. Browser isolation/partitioning and deployment headers affect whether the bridges can reach the same worker. Use the configured localhost setup for the demonstrated behavior. Pending disables a client's controls but does not globally lock other tabs. UI roles can be bypassed by custom browser code. Full snapshots and cloning are simple for a small list but copy more data as the list grows.

Possible future changes, not current functionality

Production deployment would need configurable origins/asset URLs, compatible CORS and frame/worker policies, and stable exposure contracts. Durable or multi-user state would require an appropriate persistence/network service. Conflict protection could add revisions; reliable retries could add deduplication; preserving failed drafts could wait for confirmation before clearing forms. These are design extensions, not features already present in the source.

14 | A practical code walkthrough and debugging route

Recommended presentation sequence

Begin with A/B/C open. Explain their visible responsibilities. Add 'Prepare demo' in A and show all lists. Edit it in B and show A/C change. Complete it in A. Increment B twice and decrement C once, then point to A's live summary. Delete the task in C. Add another task and reload A while B/C stay open to demonstrate recovery from the live worker.

Then show the code in dependency order: A's federation.config.js, B's loadRemoteModule call, A's exported appState instance, the client's constructor and send/receive, the bridge's connect and relay, the worker's store/applyAction/onconnect, and finally AsyncPipe in the templates. This makes each file answer the question raised by the previous one.

Breakpoints that reveal the complete path

Place a breakpoint in B.increment, AppStateClient.send, the bridge window message listener, worker applyAction, the store subscription, client receive and publish. The worker and iframe have separate debugging contexts; a breakpoint in the top-level page alone will not expose all worker locals. Use browser developer tools to select the relevant frame or inspect the shared worker.

At send, inspect requestId and action. At applyAction, inspect store.value before the clone and state before store.next. In receive, compare a state message with done and watch pendingRequest. This proves why data updates and action acknowledgements are distinct.

Network evidence

Reload B with Network open. Locate A's remoteEntry.json and the generated AppState module/dependency assets. Then locate state-bridge.html, state-bridge.js, state-worker.js and the RxJS UMD asset, allowing for browser cache and worker reuse. Their request order can overlap; use their purpose rather than promising a fixed waterfall count.

Now increment or edit. There should be no custom todo/counter HTTP API request triggered by these actions. Browser messages are not ordinary HTTP requests in the Network panel. Development tooling can maintain live-reload connections; those are separate from business-state communication.

Questions and accurate answers

Where is the actual state? In the BehaviorSubject created in Project-A/public/state-worker.js. Where is federation used? In federation.config.js to expose the client, main.ts to initialize the environment, and B/C ngOnInit to load A's entry URL and ./AppState exposure.

Is app-state.service.ts the common store? It is the federated page client and received-view cache. Why duplicate app-state.ts in B/C? Those are type-only contracts, not implementations. Why the bridge? To give every tab an A-origin document that can connect to the same worker. Why RxJS? To retain/publish the worker snapshot and expose reactive page updates to Angular.

Can B run without A's tab? Yes, if A's assets are available and the worker connection can be created. Can B load without A's asset server? A fresh load needs the exposed client and bridge assets. Does singleton share across tabs? No. Does data persist after closing everything? No. Does this demonstrate a backend microservice? No, it demonstrates frontend federation and browser-owned shared state.

15 | Complete source reference and reading instructions

The appendix includes all custom executable state/client/bridge/feature files and the relevant federation/startup files. Listings are read directly from the workspace during PDF generation and carry original line numbers. Long lines wrap visually; a wrapped continuation does not represent an extra source line. Unicode UI punctuation is normalized to ASCII in printed listings for reliable presentation.

The source is grouped by execution responsibility rather than alphabetical order. A's startup pattern is shared by B/C; the root selectors and feature classes differ. Standard CSS, dependency lockfiles and complete generated build artifacts are not repeated because they do not add business-flow logic. The build/asset settings and each dependency's role are explained in chapters 2-4.

Use the earlier chapters as the explanation beside these listings: chapter 6 covers every client method; chapter 8 covers every bridge handler; chapter 9 covers every worker function; chapter 10 covers the component methods and template bindings. Chapters 11-13 trace the same code through concrete success, concurrency and failure scenarios.

A1 | Federated client

Project-A/src/app/app-state.service.ts

```ts
import { Observable, shareReplay } from 'rxjs';

export interface TodoItem {
  id: number;
  text: string;
  completed: boolean;
}

export interface AppState {
  countB: number;
  countC: number;
  todos: TodoItem[];
  connected: boolean;
  pending: boolean;
  error: string;
}

type StateAction =
  | { type: 'counter'; project: 'b' | 'c'; action: 'increment' | 'decrement' | 'reset' }
  | { type: 'addTodo'; text: string }
  | { type: 'toggleTodo'; id: number }
  | { type: 'editTodo'; id: number; text: string }
  | { type: 'deleteTodo'; id: number };

const projectAOrigin = 'http://localhost:4200';
const channel = 'project-a-state';

// This client sends actions to the one RxJS store in state-worker.js.
class AppStateClient {
  private view: AppState = {
    countB: 0, countC: 0, todos: [],
    connected: false, pending: false, error: '',
  };
  private emit?: (view: AppState) => void;
  // Share one connection and give each Angular view the latest received value.
  readonly state$ = new Observable<AppState>(subscriber => {
    this.emit = view => subscriber.next(view);
    subscriber.next(this.view);
    return () => { this.emit = undefined; };
  }).pipe(shareReplay({ bufferSize: 1, refCount: false }));

  private bridge: HTMLIFrameElement;
  private replyTimeout?: ReturnType<typeof setTimeout>;
  private pendingRequest?: string;

  constructor() {
    this.bridge = document.createElement('iframe');
    this.bridge.hidden = true;
    this.bridge.title = 'Project A shared store connection';
    this.bridge.src = `${projectAOrigin}/state-bridge.html`;
    this.bridge.onload = () => {
      this.bridge.contentWindow?.postMessage({ channel, type: 'connect' }, projectAOrigin);
    };
    window.addEventListener('message', this.receive);
    document.body.appendChild(this.bridge);
    this.waitForReply();
  }

  updateCounter(project: 'b' | 'c', action: 'increment' | 'decrement' | 'reset') {
    return this.send({ type: 'counter', project, action });
  }

  addTodo(text: string) {
    return this.send({ type: 'addTodo', text });
  }

  toggleTodo(id: number) {
    return this.send({ type: 'toggleTodo', id });
```

```
  }

  editTodo(id: number, text: string) {
    return this.send({ type: 'editTodo', id, text });
  }

  deleteTodo(id: number) {
    return this.send({ type: 'deleteTodo', id });
  }

  private publish(view: AppState) {
    this.view = view;
    this.emit?.(view);
  }

  private send(action: StateAction) {
    if (!this.view.connected || this.view.pending) return false;
    this.pendingRequest = crypto.randomUUID();
    this.publish({ ...this.view, pending: true, error: '' });
    this.waitForReply();
    this.bridge.contentWindow?.postMessage({
      channel, type: 'action', requestId: this.pendingRequest, action,
    }, projectAOrigin);
    return true;
  }

  private receive = (event: MessageEvent) => {
    if (event.origin !== projectAOrigin || event.source !== this.bridge.contentWindow) return;
    const message = event.data;
    if (message?.channel !== channel) return;

    if (message.type === 'state' && this.validState(message.state)) {
      // Every tab receives data updates. This does not finish our pending action.
      this.publish({ ...message.state, connected: true, pending: this.view.pending, error: ''
});
      if (!this.pendingRequest) clearTimeout(this.replyTimeout);
    } else if (message.type === 'done' && message.requestId === this.pendingRequest) {
      clearTimeout(this.replyTimeout);
      this.pendingRequest = undefined;
      this.publish({ ...this.view, pending: false });
    } else if (message.type === 'error' &&
               (message.fatal || message.requestId === this.pendingRequest)) {
      const error = typeof message.message === 'string' ? message.message : 'Update failed.';
      this.showError(error, Boolean(message.fatal));
    }
  };

  private validState(state: AppState | undefined) {
    return state && Number.isSafeInteger(state.countB) && Number.isSafeInteger(state.countC)
&&
      Array.isArray(state.todos) && state.todos.every(task =>
        task && Number.isSafeInteger(task.id) && typeof task.text === 'string' &&
        typeof task.completed === 'boolean');
  }

  private showError(error: string, disconnected: boolean) {
    clearTimeout(this.replyTimeout);
    this.pendingRequest = undefined;
    this.publish({
      ...this.view, connected: disconnected ? false : this.view.connected, pending: false,
error,
    });
  }

  private waitForReply() {
    clearTimeout(this.replyTimeout);
    this.replyTimeout = setTimeout(() => {
      this.showError('No reply from the shared store. Start Project A on port 4200 and
reload.', true);
    }, 10000);
  }
}

export const appState = new AppStateClient();
```

A2 | Browser bridge

Project-A/public/state-bridge.js

```
// All three tabs embed this A-origin page to reach one SharedWorker.
const channel = 'project-a-state';
const allowedOrigins = new Set([
  'http://localhost:4200', 'http://localhost:4201', 'http://localhost:4202',
]);
let parentOrigin;
let worker;

function sendToParent(message) {
  if (parentOrigin) window.parent.postMessage({ channel, ...message }, parentOrigin);
}

function connectionError() {
  sendToParent({
    type: 'error', fatal: true,
    message: 'Shared store connection failed. Use Chrome or Edge and reload the apps.',
  });
}

function connect() {
  try {
    worker = new SharedWorker('/state-worker.js', 'project-a-rxjs-store');
    worker.onerror = connectionError;
    worker.port.onmessage = event => sendToParent(event.data);
    worker.port.onmessageerror = connectionError;
    worker.port.start();
  } catch {
    connectionError();
  }
}

window.addEventListener('message', event => {
  if (event.source !== window.parent || !allowedOrigins.has(event.origin)) return;
  const message = event.data;
  if (message?.channel !== channel) return;
  parentOrigin = event.origin;
  if (message.type === 'connect' && !worker) connect();
  if (message.type === 'action' && worker) worker.port.postMessage(message);
});

window.addEventListener('pagehide', () => {
  if (worker) {
    worker.port.postMessage({ type: 'disconnect' });
    worker.port.close();
    worker = undefined;
  }
});

// Reconnect when the browser restores a page from its back/forward cache.
window.addEventListener('pageshow', event => {
  if (event.persisted && parentOrigin && !worker) connect();
});
```

A3 | Shared RxJS worker

Project-A/public/state-worker.js

```
importScripts('/worker/rxjs.umd.min.js');

// One store for every connected application tab.
const store = new rxjs.BehaviorSubject({ countB: 0, countC: 0, todos: [] });
const clients = new Set();
let nextTodoId = 1;

store.subscribe(state => {
  for (const client of clients) {
    try {
      client.postMessage({ type: 'state', state });
    } catch {
      clients.delete(client);
    }
  }
});

function taskText(text) {
  if (typeof text !== 'string' || !text.trim()) throw new Error('Task text cannot be blank.');
  return text.trim();
}

function findTask(state, id) {
  const task = state.todos.find(task => task.id === id);
  if (!task) throw new Error('This task no longer exists. Use the updated list.');
  return task;
}

function updateCounter(state, action) {
  if (!['b', 'c'].includes(action.project) ||
      !['increment', 'decrement', 'reset'].includes(action.action)) {
    throw new Error('Invalid counter action.');
  }
  const key = action.project === 'b' ? 'countB' : 'countC';
  if (action.action === 'increment') state[key]++;
  if (action.action === 'decrement') state[key]--;
  if (action.action === 'reset') state[key] = 0;
  if (!Number.isSafeInteger(state[key])) throw new Error('Counter limit reached.');
}

function applyAction(action) {
  if (!action) throw new Error('Invalid action.');

  // Work on a copy, then publish the result once.
  const state = structuredClone(store.value);

  switch (action.type) {
    case 'counter':
      updateCounter(state, action);
      break;
    case 'addTodo': {
      const text = taskText(action.text);
      state.todos.push({ id: nextTodoId++, text, completed: false });
      break;
    }
    case 'toggleTodo': {
      const task = findTask(state, action.id);
      task.completed = !task.completed;
      break;
    }
    case 'editTodo':
      findTask(state, action.id).text = taskText(action.text);
      break;
    case 'deleteTodo':
      findTask(state, action.id);
      state.todos = state.todos.filter(task => task.id !== action.id);
      break;
    default:
```

```
      throw new Error('Unknown action.');
    }

    store.next(state);
  }

  self.onconnect = event => {
    const port = event.ports[0];
    clients.add(port);

    port.onmessage = event => {
      const message = event.data;
      if (message?.type === 'disconnect') {
        clients.delete(port);
        port.close();
        if (clients.size === 0) self.close();
        return;
      }
      if (message?.type !== 'action' || typeof message.requestId !== 'string') return;

      try {
        applyAction(message.action);
        // Only the requesting tab receives this acknowledgement.
        port.postMessage({ type: 'done', requestId: message.requestId });
      } catch (error) {
        port.postMessage({ type: 'error', requestId: message.requestId, message: error.message
});
      }
    };

    port.start();
    port.postMessage({ type: 'state', state: store.value });
  };
```

A4 | Project A components

Project-A/src/app/app.ts

```ts
import { AsyncPipe } from '@angular/common';
import { Component } from '@angular/core';
import { appState } from './app-state.service';
import { Todo } from './todo';

@Component({
  selector: 'app-root',
  imports: [AsyncPipe, Todo],
  styleUrl: './app.css',
  templateUrl: './app.html',
})
export class App {
  state$ = appState.state$;
}
```

Project-A/src/app/todo.ts

```ts
import { AsyncPipe } from '@angular/common';
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { appState } from './app-state.service';

@Component({
  selector: 'app-todo',
  imports: [AsyncPipe, FormsModule],
  templateUrl: './todo.html',
  styleUrl: './todo.css',
})
export class Todo {
  state$ = appState.state$;
  newTask = '';

  addTask() {
    if (appState.addTodo(this.newTask)) this.newTask = '';
  }

  toggleTask(id: number) {
    appState.toggleTodo(id);
  }
}
```

A5 | Project A templates

Project-A/src/app/app.html

```
<header>
  <h1>Project A</h1>
  <p>One common RxJS store for Project A, B, and C in separate tabs.</p>
  <div class="open-projects">
    <a href="http://localhost:4201" target="_blank" rel="noopener">Open Project B in a new tab</a>
    <a href="http://localhost:4202" target="_blank" rel="noopener">Open Project C in a new tab</a>
  </div>
</header>

<main>
  @if (state$ | async; as state) {
    <section class="counter-summary" aria-label="Live counter values">
      <h2>Live counters</h2>
      <p aria-live="polite">Project B: <strong>{{ state.connected ? state.countB : '-' }}</strong></p>
      <p aria-live="polite">Project C: <strong>{{ state.connected ? state.countC : '-' }}</strong></p>
      @if (state.error) {
        <p role="alert">{{ state.error }}</p>
      } @else if (!state.connected) {
        <p>Connecting to the shared RxJS store...</p>
      }
      <p>Data stays in browser memory while a connected application tab remains open.</p>
    </section>
  }
  <section class="todo-panel" aria-label="Project A tasks">
    <app-todo />
  </section>
</main>
```

Project-A/src/app/todo.html

```
<section>
  <h2>Todo list</h2>
  <p>Add and complete tasks here. Edit task text in Project B and delete tasks in Project C.</p>

  @if (state$ | async; as state) {
    <form (ngSubmit)="addTask()">
      <label for="new-task">New task</label>
      <div class="task-input">
        <input id="new-task" name="newTask" [(ngModel)]="newTask" placeholder="Enter a task" />
        <button type="submit" [disabled]="!newTask.trim() || !state.connected || state.pending">Add</button>
      </div>
    </form>
    <ul aria-label="Todo tasks">
      @for (task of state.todos; track task.id) {
        <li>
          <label class="task">
            <input type="checkbox" [checked]="task.completed" (change)="toggleTask(task.id)"
              [disabled]="!state.connected || state.pending" />
            <span [class.completed]="task.completed">{{ task.text }}</span>
          </label>
        </li>
      } @empty {
        <li>{{ state.connected ? 'No tasks yet. Add your first task above.' : 'Loading tasks...' }}</li>
      }
    </ul>
  }
</section>
```

A6 | Project B component

Project-B/src/app/app.ts

```ts
import { AsyncPipe } from '@angular/common';
import { Component, OnInit, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { loadRemoteModule } from '@angular-architects/native-federation';
import type { AppStateService, TodoItem } from './app-state';

@Component({
  selector: 'app-counter-b',
  imports: [AsyncPipe, FormsModule],
  templateUrl: './app.html',
  styleUrl: './app.css',
})
export class App implements OnInit {
  appState = signal<AppStateService | null>(null);
  error = signal('');
  editingId: number | null = null;
  draftText = '';

  async ngOnInit() {
    try {
      const module = await loadRemoteModule({
        remoteEntry: 'http://localhost:4200/remoteEntry.json',
        exposedModule: './AppState',
      });
      this.appState.set(module.appState);
    } catch {
      this.error.set('Could not load the shared state client. Start Project A on port 4200 and reload.');
    }
  }

  increment() {
    this.appState()?.updateCounter('b', 'increment');
  }

  decrement() {
    this.appState()?.updateCounter('b', 'decrement');
  }

  reset() {
    this.appState()?.updateCounter('b', 'reset');
  }

  startEditing(task: TodoItem) {
    this.editingId = task.id;
    this.draftText = task.text;
  }

  saveTask(id: number) {
    const text = this.draftText.trim();
    if (text && this.appState()?.editTodo(id, text)) this.cancelEditing();
  }

  cancelEditing() {
    this.editingId = null;
    this.draftText = '';
  }
}
```

A7 | Project B template and contract

Project-B/src/app/app.html

```
<h1 class="project-title">Project B</h1>

@if (appState(); as service) {
  @if (service.state$ | async; as state) {
    <section aria-label="Project B counter">
      <h2>Project B counter</h2>
      <p class="count" aria-live="polite">{{ state.connected ? state.countB : '-' }}</p>
      <div class="buttons">
        <button type="button" (click)="decrement()" [disabled]="!state.connected || state.pending"
          aria-label="Decrease count">-</button>
        <button type="button" (click)="reset()" [disabled]="!state.connected || state.pending">Reset</button>
        <button type="button" (click)="increment()" [disabled]="!state.connected || state.pending"
          aria-label="Increase count">+</button>
      </div>
    </section>

    <section aria-label="Edit shared tasks">
      <h2>Edit tasks</h2>
      <p>Edit the text of tasks created in Project A. Delete tasks in Project C.</p>
      <ul aria-label="Todo tasks">
        @for (task of state.todos; track task.id) {
          <li>
            @if (editingId === task.id) {
              <form (ngSubmit)="saveTask(task.id)" class="edit-form">
                <label [for]="'edit-task-' + task.id">Task text</label>
                <input [id]="'edit-task-' + task.id" name="taskText" [(ngModel)]="draftText" />
                <div class="buttons">
                  <button type="submit" [disabled]="!draftText.trim() || !state.connected || state.pending">Save</button>
                  <button type="button" (click)="cancelEditing()">Cancel</button>
                </div>
              </form>
            } @else {
              <span [class.completed]="task.completed">{{ task.text }}</span>
              <button type="button" (click)="startEditing(task)" [disabled]="!state.connected || state.pending"
                [attr.aria-label]="'Edit ' + task.text">Edit</button>
            }
          </li>
        } @empty {
          <li>{{ state.connected ? 'No tasks yet. Add a task in Project A.' : 'Loading tasks...' }}</li>
        }
      </ul>
    </section>

    @if (state.error) {
      <p class="status" role="alert">{{ state.error }}</p>
    } @else if (!state.connected) {
      <p class="status">Connecting to the common RxJS store...</p>
    }
  }
} @else if (error()) {
  <p class="status" role="alert">{{ error() }}</p>
} @else {
  <p class="status">Loading the federated state client...</p>
}
```

Project-B/src/app/app-state.ts

```ts
import type { Observable } from 'rxjs';

export interface TodoItem {
  id: number;
  text: string;
  completed: boolean;
}

export interface AppState {
  countB: number;
  countC: number;
  todos: TodoItem[];
  connected: boolean;
  pending: boolean;
  error: string;
}

// A type-only contract for the federated client; no local store is created.
export interface AppStateService {
  readonly state$: Observable<AppState>;
  updateCounter(project: 'b' | 'c', action: 'increment' | 'decrement' | 'reset'): boolean;
  editTodo(id: number, text: string): boolean;
}
```

A8 | Project C component

Project-C/src/app/app.ts

```ts
import { AsyncPipe } from '@angular/common';
import { Component, OnInit, signal } from '@angular/core';
import { loadRemoteModule } from '@angular-architects/native-federation';
import type { AppStateService } from './app-state';

@Component({
  selector: 'app-counter-c',
  imports: [AsyncPipe],
  templateUrl: './app.html',
  styleUrl: './app.css',
})
export class App implements OnInit {
  appState = signal<AppStateService | null>(null);
  error = signal('');

  async ngOnInit() {
    try {
      const module = await loadRemoteModule({
        remoteEntry: 'http://localhost:4200/remoteEntry.json',
        exposedModule: './AppState',
      });
      this.appState.set(module.appState);
    } catch {
      this.error.set('Could not load the shared state client. Start Project A on port 4200 and reload.');
    }
  }

  increment() {
    this.appState()?.updateCounter('c', 'increment');
  }

  decrement() {
    this.appState()?.updateCounter('c', 'decrement');
  }

  reset() {
    this.appState()?.updateCounter('c', 'reset');
  }

  deleteTask(id: number) {
    this.appState()?.deleteTodo(id);
  }
}
```

A9 | Project C template and contract

Project-C/src/app/app.html

```
<h1 class="project-title">Project C</h1>

@if (appState(); as service) {
  @if (service.state$ | async; as state) {
    <section aria-label="Project C counter">
      <h2>Project C counter</h2>
      <p class="count" aria-live="polite">{{ state.connected ? state.countC : '-' }}</p>
      <div class="buttons">
        <button type="button" (click)="decrement()" [disabled]="!state.connected ||
state.pending"
          aria-label="Decrease count">-</button>
        <button type="button" (click)="reset()" [disabled]="!state.connected ||
state.pending">Reset</button>
        <button type="button" (click)="increment()" [disabled]="!state.connected ||
state.pending"
          aria-label="Increase count">+</button>
      </div>
    </section>

    <section aria-label="Delete shared tasks">
      <h2>Delete tasks</h2>
      <p>Delete tasks created in Project A. Edit task text in Project B.</p>
      <ul aria-label="Todo tasks">
        @for (task of state.todos; track task.id) {
          <li>
            <span [class.completed]="task.completed">{{ task.text }}</span>
            <button type="button" (click)="deleteTask(task.id)" [disabled]="!state.connected
|| state.pending"
              [attr.aria-label]="'Delete ' + task.text">Delete</button>
          </li>
        } @empty {
          <li>{{ state.connected ? 'No tasks yet. Add a task in Project A.' : 'Loading
tasks...' }}</li>
        }
      </ul>
    </section>

    @if (state.error) {
      <p class="status" role="alert">{{ state.error }}</p>
    } @else if (!state.connected) {
      <p class="status">Connecting to the common RxJS store...</p>
    }
  }
} @else if (error()) {
  <p class="status" role="alert">{{ error() }}</p>
} @else {
  <p class="status">Loading the federated state client...</p>
}
```

Project-C/src/app/app-state.ts

```ts
import type { Observable } from 'rxjs';

export interface TodoItem {
  id: number;
  text: string;
  completed: boolean;
}

export interface AppState {
  countB: number;
  countC: number;
  todos: TodoItem[];
  connected: boolean;
  pending: boolean;
  error: string;
}

// A type-only contract for the federated client; no local store is created.
export interface AppStateService {
  readonly state$: Observable<AppState>;
  updateCounter(project: 'b' | 'c', action: 'increment' | 'decrement' | 'reset'): boolean;
  deleteTodo(id: number): boolean;
}
```

A10 | Consumer federation configurations

Project-B/federation.config.js

```
const { withNativeFederation, shareAll } = require('@angular-architects/native-federation/config');

module.exports = withNativeFederation({
  name: 'project-b',
  features: {
    ignoreUnusedDeps: false,
  },

  shared: {
    ...shareAll({ singleton: true, strictVersion: true, requiredVersion: 'auto' }),
  },

  skip: [
    '@angular/platform-browser/animations',
    '@angular/platform-browser/animations/async',
    'rxjs/ajax',
    'rxjs/fetch',
    'rxjs/testing',
    'rxjs/webSocket',
  ],
});
```

Project-C/federation.config.js

```
const { withNativeFederation, shareAll } = require('@angular-architects/native-federation/config');

module.exports = withNativeFederation({
  name: 'project-c',
  features: {
    ignoreUnusedDeps: false,
  },

  shared: {
    ...shareAll({ singleton: true, strictVersion: true, requiredVersion: 'auto' }),
  },

  skip: [
    '@angular/platform-browser/animations',
    '@angular/platform-browser/animations/async',
    'rxjs/ajax',
    'rxjs/fetch',
    'rxjs/testing',
    'rxjs/webSocket',
  ],
});
```

16 | References and verification scope

The application-specific explanation is based on the current Project-A, Project-B and Project-C source, their package/build/federation configuration, and the installed Native Federation adapter and RxJS source. The PDF generation does not modify application behavior. Source listings are a snapshot of the files at generation time, not a promise that later edits retain the same line numbers.

For the adapter details, the inspected dependency is Project-A/node_modules/@angular-architects/native-federation/src/index.js. For BehaviorSubject and shareReplay semantics, the inspected files are Project-A/node_modules/rxjs/src/internal/BehaviorSubject.ts and src/internal/operators/shareReplay.ts. These implementation reads explain the distinction between current-value storage, synchronous notification and local replay.

Official documentation for further reading

Angular AsyncPipe: https://angular.dev/api/common/AsyncPipe

RxJS subjects and BehaviorSubject: https://rxjs.dev/guide/subject

RxJS shareReplay: https://rxjs.dev/api/operators/shareReplay

Native Federation project documentation: https://github.com/angular-architects/module-federation-plugin/tree/main/libs/native-federation

SharedWorker and same-origin access: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

Window messaging and origin validation: https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage

The local project remains the source of truth for which features are actually implemented. Browser and library documentation explain platform concepts; they should not be read as evidence that this demo implements persistence, retries, authentication or remote component embedding.